LAPORAN RESMI PRAKTIKUM

DevSecOps — Bab 3

Docker Network, Volume, Bind Mount, tmpfs, dan Compose

Nama : Fawwaz Mufid Wardaya
Kelas : D4 Teknik Informatika B
NRP : 3126640050

PROGRAM STUDI TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

Pendahuluan

Latar Belakang

Containerization memisahkan aplikasi dari infrastruktur, tetapi aplikasi nyata tidak berdiri sendiri: ia berkomunikasi dengan layanan lain dan menyimpan state. Bab 3 membahas tiga dimensi yang sering dicampur, yaitu network (siapa yang dapat berbicara dengan siapa), storage (di mana state hidup dan berapa lama bertahan), dan Compose (bagaimana komponen serta resource dinyatakan sebagai satu aplikasi deklaratif). Praktikum ini membangun user-defined bridge network beserta pembuktian resolusi DNS, menguji persistensi named volume berikut backup dan restore, membandingkannya dengan tmpfs yang bersifat sementara, lalu menjalankan stack tiga lapis Nginx, Flask, dan PostgreSQL dengan Docker Compose.

Rumusan Masalah

Mengapa user-defined bridge lebih baik daripada default bridge untuk aplikasi multi-container?

Apa risiko bind mount terhadap keamanan host?

Apa perbedaan docker compose down dan docker compose down -v?

Kapan depends_on dengan healthcheck lebih tepat daripada depends_on biasa?

Bagaimana strategi backup volume untuk database produksi?

Pembahasan

Dasar Teori

Jaringan container membentuk graf keterjangkauan. Dua service dapat berkomunikasi bila berbagi network. User-defined bridge menyediakan resolusi DNS otomatis berdasarkan nama service, sehingga aplikasi memakai nama logis (db:5432, app:5000) alih-alih alamat IP yang dapat berubah. Publikasi port dengan -p HOST:CONTAINER hanya diperlukan pada ingress; tanpa pembatasan alamat bind, port terbuka pada seluruh interface host. Prinsip least exposure menyarankan hanya reverse proxy atau API publik yang dipublikasikan.

Untuk data, writable layer container bersifat sementara. Tiga mekanisme mount menyediakan lifecycle berbeda:

Tipe mount	Lokasi	Persisten	Kapan digunakan
Volume	Dikelola Docker (/var/lib/docker/volumes)	Ya	Database, data aplikasi
Bind mount	Path host yang ditentukan user	Ya	Live development, file konfigurasi
tmpfs	RAM host	Tidak	Cache sementara, data sensitif sementara

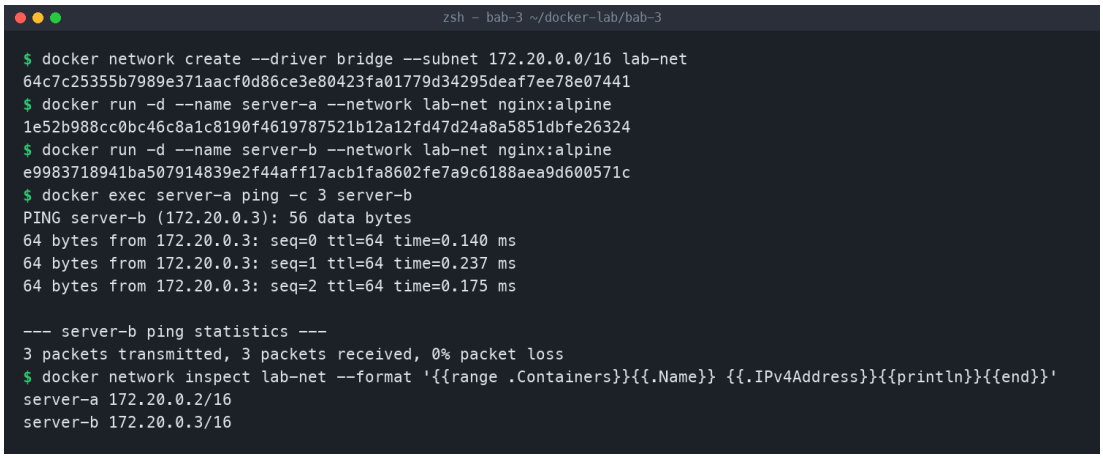
Docker Compose menyatakan model multi-container dalam YAML. Field top-level networks dan volumes dideklarasikan lalu diberikan ke service. depends_on mengatur urutan start, tetapi tidak menjamin kesiapan; condition service_healthy membuat Compose menunggu healthcheck dependency. Compose juga memberi nama resource berdasarkan project, misalnya network bab-3_frontend dan volume bab-3_pg-data.

Praktikum 1 — User-defined bridge network dan resolusi DNS

Perintah:

```
docker network create --driver bridge --subnet 172.20.0.0/16 lab-net
docker run -d --name server-a --network lab-net nginx:alpine
```

```
docker run -d --name server-b --network lab-net nginx:alpine
docker exec server-a ping -c 3 server-b
```



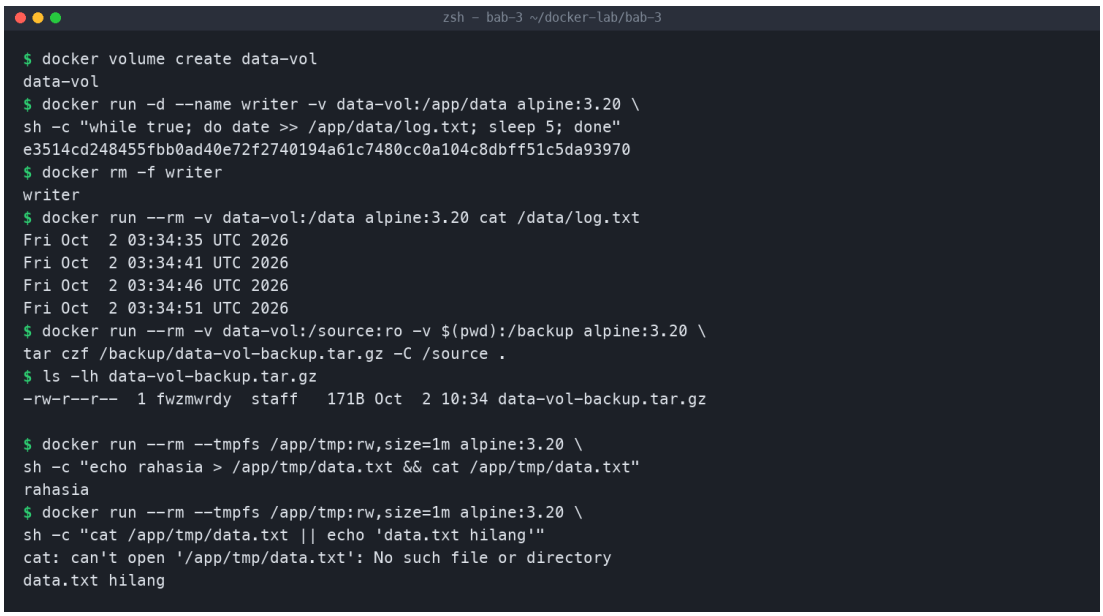
Gambar 1. server-a melakukan ping ke server-b melalui nama service pada user-defined bridge.

Analisis: server-a berhasil me-resolve nama server-b menjadi 172.20.0.3 tanpa entri manual, dan 3 paket ICMP diterima tanpa kehilangan. Pembuktian ini menegaskan bahwa user-defined bridge menyediakan embedded DNS berbasis nama, sesuatu yang tidak dimiliki default bridge. Nama service menjadi identitas logis yang stabil meskipun alamat IP container berubah ketika container dibuat ulang.

Praktikum 2 — Named volume: persistensi, backup/restore, dan tmpfs

Perintah:

```
docker volume create data-vol
docker run -d --name writer -v data-vol:/app/data alpine:3.20 \
  sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"
sleep 15 && docker rm -f writer
docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
docker run --rm -v data-vol:/source:ro -v "$(pwd)":/backup alpine:3.20 \
  tar czf /backup/data-vol-backup.tar.gz -C /source .
```



Gambar 2. Data named volume bertahan setelah container dihapus; tmpfs hilang pada container berikutnya.

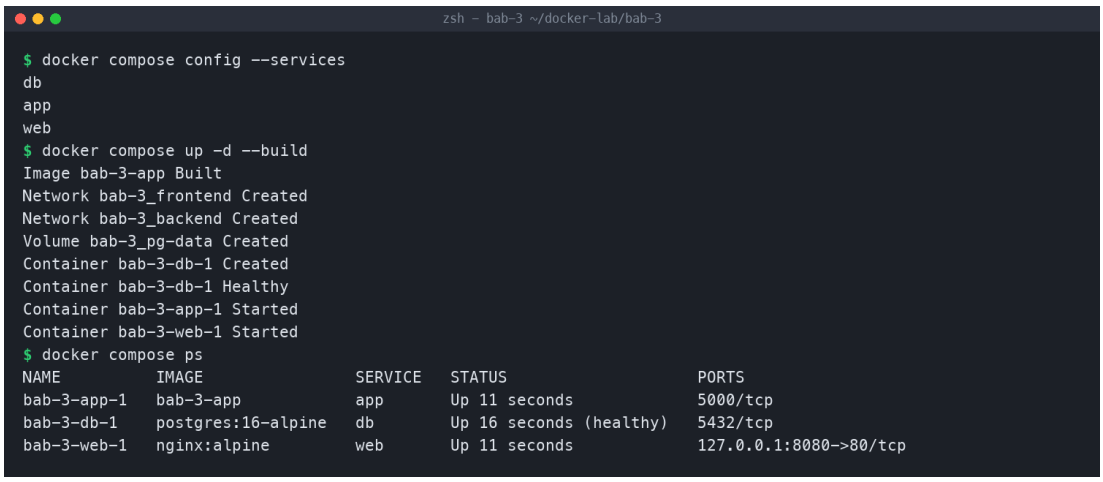
Analisis: Empat baris log tetap terbaca setelah container writer dihapus dengan docker rm -f, membuktikan named volume hidup melampaui lifecycle container. Backup dibuat dengan memount volume sebagai read-only (:ro) dan menyimpan arsip ke host melalui bind mount. Sebaliknya, tmpfs

menampilkan data pada container pertama lalu gagal membuka file pada container berikutnya, menegaskan bahwa tmpfs hanya cocok untuk data temporer. Catatan operasional: volume yang persisten tidak otomatis terlindungi; ia tetap dapat terhapus oleh prune atau down -v.

Praktikum 3 — Stack Compose Nginx–Flask–PostgreSQL dan bind mount

Perintah:

```
cd ~/docker-lab/bab-3
docker compose config --services
docker compose up -d --build
docker compose ps
curl -i http://localhost:8080/
curl -s http://localhost:8080/health
```



Gambar 3. Stack tiga lapis berjalan; hanya service web yang mempublikasikan port.



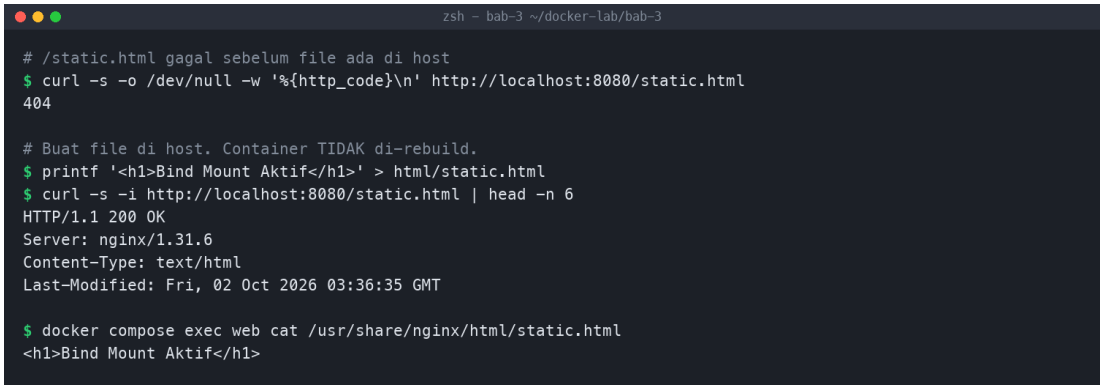
Gambar 4. Endpoint / mengembalikan versi PostgreSQL dan /health mengembalikan status healthy.

Analisis: Karena db memakai depends_on dengan condition service_healthy, Compose menahan start app sampai PostgreSQL lulus pg_isready, sehingga tidak muncul error koneksi saat boot. Nginx meneruskan permintaan ke app:5000 dan aplikasi merespons dengan versi PostgreSQL 16.15, membuktikan rantai web ke app ke db utuh. Dari sisi keamanan: hanya web yang terpublikasi (127.0.0.1:8080), sedangkan app dan db hanya memiliki port internal (5000/tcp, 5432/tcp). Aplikasi juga berjalan sebagai non-root (uid=10001 appuser), dan network terpisah menempatkan db hanya pada backend sehingga tidak dapat dijangkau langsung oleh web.

Analisis Masalah: /static.html Mengembalikan 404

Saat pertama diuji, http://localhost:8080/static.html mengembalikan 404 Not Found. Diagnosa dilakukan berlapis: docker compose ps menunjukkan web berjalan, lalu log Nginx menampilkan pesan "open() /usr/share/nginx/html/static.html failed (2: No such file or directory)". Ternyata bind mount hanya berisi index.html, sedangkan konfigurasi Nginx mencari static.html. Masalah ini adalah

kesalahan konten pada direktori bind mount, bukan kegagalan container. Setelah file static.html dibuat di host, permintaan langsung mengembalikan 200 tanpa rebuild image, sekaligus membuktikan perilaku live-reload bind mount.



Gambar 5. Perbaikan 404 menjadi 200 setelah file dibuat di host, tanpa rebuild image.

Troubleshooting

Gejala	Penyebab yang mungkin	Tindakan korektif
/static.html 404	Bind mount tidak memuat static.html	Buat file di host; bind mount langsung aktif
Database unhealthy	Password atau healthcheck salah	docker compose logs db; cek pg_isready
502 Bad Gateway	Aplikasi belum siap saat proxy menerima request	Gunakan depends_on condition service_healthy
YAML gagal diproses	Tab atau indentasi salah	docker compose config untuk melihat model akhir
port is already allocated	Port host 8080 dipakai proses lain	Ganti ke 127.0.0.1:8081:80

Evaluasi dan Latihan Mandiri

1. Mengapa user-defined bridge lebih baik daripada default bridge untuk multi-container app?

User-defined bridge menyediakan embedded DNS sehingga container dapat saling memanggil melalui nama service, bukan alamat IP yang dapat berubah. Isolasi juga lebih tegas: hanya container pada network yang sama yang dapat berkomunikasi, sehingga segmentasi frontend dan backend dapat dibentuk. Default bridge tidak menyediakan resolusi nama otomatis dan lebih rentan terhadap komunikasi yang tidak diinginkan.

2. Apa risiko bind mount terhadap keamanan host?

Bind mount memetakan path host langsung ke container dengan akses tulis secara default. Proses di container dapat mengubah atau menghapus file host, sehingga bind mount yang longgar (misalnya /etc atau socket Docker) memperluas attack surface hingga berpotensi setara akses root host. Mitigasi: gunakan opsi :ro bila tulis tidak diperlukan, batasi path yang di-mount, dan hindari memberi socket Docker ke workload yang tidak dipercaya.

3. Apa perbedaan docker compose down dan docker compose down -v?

docker compose down menghentikan dan menghapus container serta network project, tetapi mempertahankan named volume. docker compose down -v juga menghapus volume yang terkait, sehingga bersifat destruktif terhadap data. Pada praktikum ini, down biasa masih menyisakan bab-3_pg-data, sedangkan down -v akan menghapusnya.

4. Kapan depends_on dengan healthcheck lebih tepat daripada depends_on biasa?

depends_on biasa hanya mengatur urutan start dan tidak menjamin dependency siap. Ketika aplikasi bergantung pada layanan yang butuh waktu inisialisasi (seperti PostgreSQL yang melakukan recovery atau migration), condition service_healthy lebih tepat karena Compose menunggu healthcheck lulus sebelum memulai service yang bergantung padanya. Aplikasi tetap disarankan memiliki retry dengan backoff karena dependency dapat gagal kembali.

5. Bagaimana strategi backup volume untuk database produksi?

Backup harus konsisten secara aplikasi. Untuk PostgreSQL, gunakan pg_dump dari dalam container agar snapshot transaksional, bukan menyalin file mentah saat database menulis. Simpan hasil ke storage di luar host, terapkan retensi dan enkripsi, uji restore secara berkala, dan monitor kapasitas. Pada praktikum, pola tar terhadap mount :ro menunjukkan mekanisme dasar, tetapi untuk produksi dump logis tetap lebih aman.

Penutup

Kesimpulan

Praktikum Bab 3 berhasil membuktikan tiga dimensi Docker. Network: user-defined bridge lab-net menyediakan resolusi DNS sehingga server-a dapat melakukan ping ke server-b melalui nama. Storage: named volume data-vol mempertahankan data setelah container dihapus dan dapat di-backup, sementara tmpfs kehilangan data pada container berikutnya. Compose: stack Nginx–Flask–PostgreSQL berjalan dengan urutan start yang benar berkat healthcheck, dan endpoint / maupun /health merespons sesuai harapan. Dari sisi keamanan, hanya ingress web yang dipublikasikan pada 127.0.0.1, aplikasi berjalan sebagai non-root, dan database terisolasi pada network backend. Kendala 404 pada /static.html diselesaikan dengan memperbaiki isi bind mount di host, memperkuat pemahaman bahwa kesalahan konfigurasi dan kesalahan konten adalah dua hal berbeda saat mendiagnosis stack Compose.

Daftar Pustaka

Materi Ajar DevOps (DevSecOps Terintegrasi) — Bab 3. Politeknik Elektronika Negeri Surabaya, 2026.

Docker Inc. (2026). Docker Documentation — Networking, Volumes, dan Compose. https://docs.docker.com/

Docker Inc. (2026). Compose Specification. https://docs.docker.com/compose/

Akses terakhir: 2 Oktober 2026.